DeepWalk and node2vec: A From-Scratch Implementation

An implementation of two foundational graph representation learning algorithms, DeepWalk (Perozzi et al., 2014) and node2vec (Grover and Leskovec, 2016), built entirely from first principles using NumPy and PyTorch. The goal of this project is transparency: every component of both algorithms, including random walk generation, skip-gram context construction, hierarchical softmax, negative sampling, and biased second-order walks, is implemented as an explicit, inspectable function rather than invoked from an existing graph-embedding library such as `gensim` or `node2vec`.

Motivation

DeepWalk and node2vec are widely used as baselines in graph representation learning, but most public implementations rely on `gensim.Word2Vec` as a black box for the optimization step. This project instead re-derives and re-implements the mechanics described in both papers, so that the relationship between graph structure, random walks, and the resulting embeddings can be examined directly at each stage of the pipeline.

Repository Structure

```
.
|-- DeepWalk_Node2Vec_Implementation.ipynb   Main notebook: full implementation, experiments
|-- data/
|   `-- Homo_sapiens.mat                     Protein-protein interaction network (node2vec p
|-- results/
|   |-- comparison.png                       2D PCA projection of DeepWalk vs. node2vec embe
|   |-- karate.png                           DeepWalk embeddings on the Zachary Karate Club
|   `-- output.png                           Precision@1 across five random seeds (robustnes
|-- rough/                                   Early exploratory notebooks and superseded draf
|-- requirements.txt
`-- README.md
```

Methods Implemented

DeepWalk - Unbiased, first-order random walks over the graph's transition matrix - Context pair generation via a sliding window over each walk - Skip-gram objective optimized with Hierarchical Softmax, implemented as an explicit binary tree over the vocabulary of nodes, matching the method described in the original paper

node2vec - Biased, second-order random walks governed by the return parameter p and the in-out parameter q - The alpha_pq(t, x) transition bias, derived and implemented directly from the paper's formulation - Alias sampling

(Walker's method) for O(1) sampling of the next walk step - Skip-gram objective optimized with Negative Sampling, as specified in the original paper

Both algorithms share the same skip-gram training loop; they differ only in how the training pairs are generated (unbiased vs. biased walks) and how the softmax is approximated (hierarchical softmax vs. negative sampling), consistent with each paper's respective design.

Experiments

1. **Toy graph baseline** — a fixed 6-node graph used to validate correctness of every function in isolation, including random walks, transition probabilities, and skip-gram training.
2. **node2vec parameter study** — walks and embeddings generated under three (p, q) settings: balanced (p=1, q=1), BFS-like (p=0.5, q=2), and DFS-like (p=2, q=0.5), evaluated for how each setting shapes the embedding space.
3. **Real-world validation on the Zachary Karate Club graph** (34 nodes, 78 edges) — the identical, unmodified pipeline applied to a real social network with a known community structure.
4. **Real-world validation on the Homo sapiens PPI network** (3,890 nodes, 38,292 edges, 50 GO-annotation labels) — the same dataset used in the original node2vec paper, evaluated with multi-label classification (macro/micro-F1) via a one-vs-rest logistic regression classifier, using a vectorized negative-sampling implementation for scalability.
5. **Multi-seed robustness check** — the full DeepWalk vs. node2vec comparison repeated across five random seeds, reporting mean and standard deviation of loss and precision@k rather than a single run.
6. **Structural equivalence correlation** — an evaluation metric independent of precision@k, measuring whether nodes of similar degree are placed close together in embedding space even when not directly connected.
7. **Hierarchical softmax and alias sampling, verified in isolation** — standalone demonstrations confirming that hierarchical softmax probabilities sum to one over the binary tree, and that alias sampling reproduces the correct biased transition distribution.

Evaluation Metrics

- **Precision@k** — for each node, the fraction of its k nearest neighbors in embedding space (by cosine similarity) that are true graph neighbors. Measures how well the embedding preserves local adjacency.
- **Structural equivalence correlation** — the correlation between embedding similarity and degree similarity across non-adjacent node pairs. Measures whether the embedding captures structural role rather than direct connectivity.
- **Macro/Micro-F1** (PPI dataset only) — standard multi-label classifica-

tion metrics used in the original node2vec paper for evaluating embedding quality on a downstream task.

Requirements

See `requirements.txt`. The notebook was developed and tested with Python 3.10+.

```
pip install -r requirements.txt
```

Usage

Open `DeepWalk_Node2Vec_Implementation.ipynb` and run all cells in order. The notebook is self-contained: the toy graph and the Zachary Karate Club edge list are defined inline, and the PPI dataset is loaded from `data/Homo_sapiens.mat`.

References

- B. Perozzi, R. Al-Rfou, and S. Skiena. "DeepWalk: Online Learning of Social Representations." KDD 2014.
- A. Grover and J. Leskovec. "node2vec: Scalable Feature Learning for Networks." KDD 2016.
- W. W. Zachary. "An Information Flow Model for Conflict and Fission in Small Groups." Journal of Anthropological Research, 1977.

Notes

The `rough/` directory contains earlier, exploratory notebooks (`Basics.ipynb`, `DeepWalk.ipynb`) written during initial development, along with a duplicate copy of the final notebook and a previously generated PDF export. These are retained for development history and are not required to reproduce the results.